From C to C++ II

Generic programming, the standard library, and callbacks

Roberto Masocco
roberto.masocco@uniroma2.it

Tor Vergata University of Rome
Department of Civil Engineering and Computer Science Engineering

April 29, 2026

Roadmap

1 Templates

2 The standard library

3 Smart pointers

4 Lambdas and callbacks

Roadmap

1 Templates

2 The standard library

3 Smart pointers

4 Lambdas and callbacks

Templates

Generic programming

Consider writing a `max()` **function** for **integers**, then again for **doubles**, then again for **any type** that supports comparison.
In **C** the only escape is a **macro** — unsafe, untyped, and undebuggable.

C++ solves this with **templates**: a mechanism for writing code that is **parameterized on a type** (or on a value).
The compiler generates the actual code for each combination of template arguments used in a program — at **compile time**, with full type checking.

Definition of template

A **template** is a blueprint for a **family** of functions or classes, parameterized by one or more **type parameters** (or non-type parameters), from which **the compiler generates concrete instantiations** on demand.

A **function template** is declared by prefixing the function with a **template parameter list**. The type parameter is conventionally named `T` (though any name works).

```
template<typename T>
T max(T a, T b)
{
  return (a > b) ? a : b;
}

int    m1 = max<int>(3, 7);          // T = int
double m2 = max<double>(3.14, 2.71); // T = double
int    m3 = max(10, 20);             // T deduced by compiler

```

Listing 1: A function template and some instantiations.

When the arguments allow it, **the compiler deduces** the template argument automatically.

A **class template** defines a family of classes.
All member functions of a class template are **implicitly templates** themselves.

```
template<typename T>
class Box
{
public:
  Box(T value) : value_(value) {}
  T get() const { return value_; }
private:
  T value_;
};
Box<int>          int_box(42);
Box<std::string> str_box("hello");

```

Listing 2: A minimal class template.

Templates

Instantiation and specialization

Each distinct set of template arguments produces a **separate instantiation** — a completely independent piece of compiled code.

A **template specialization** overrides the generic definition for that type only.

```
template<>
class Box<bool>
{
public:
  Box(bool value) : value_(value) {}
  bool get() const { return value_; }
  void flip() { value_ = !value_; }
private:
  bool value_;
};

```

Listing 3: Full specialization of `Box` for `bool`.

Templates

Taming instantiation names

Template instantiation names grow quickly and become unwieldy in real code.
`using` aliases give them shorter, meaningful names.

```
// Instead of writing this everywhere:
std::map<std::string, std::vector<int>> table;

// Define an alias:
using StringIntVecMap = std::map<std::string, std::vector<int>>;
StringIntVecMap table;

// Alias templates are also possible:
template<typename T>
using Vec = std::vector<T>;
Vec<double> data;

```

Listing 4: Type aliases for template instantiations.

Roadmap

1 Templates

2 The standard library

3 Smart pointers

4 Lambdas and callbacks

The standard library

The **C++ Standard Library** is a large collection of ready-made components, all living in the `std` namespace.
It is built almost entirely on templates, and provides

- **containers**: data structures for storing and organizing objects;
- **algorithms**: sorting, searching, transforming, accumulating, etc.;
- **utilities**: string handling, time, random numbers, type traits, etc.;
- **I/O streams**: file and console I/O;
- **concurrency**: threads, mutexes, atomics, condition variables.

For this course, a working knowledge of a small but important subset is sufficient.
The rest you will pick up as needed — the documentation is excellent.

The standard library

std::string

It manages its own memory, supports concatenation with +, comparison with ==, and all the operations you would expect; **conversion** to `const char*` is available via `s.c_str()`.

```
#include <string>

std::string s1 = "hello";
std::string s2 = "world";
std::string s3 = s1 + ", " + s2 + "!"; // "hello, world!"

std::cout << s3 << std::endl;
std::cout << s3.size() << std::endl;        // 13
std::cout << s3.substr(0, 5) << std::endl;  // "hello"

if (s1 == "hello") { /* ... */ }        // value comparison

```

Listing 5: Basic `std::string` usage.

The standard library

`std::vector`

`std::vector<T>` is a **dynamically-sized array** — the workhorse container of C++.

It stores elements **contiguously** in heap memory, **grows automatically** when needed, and provides O(1) indexed access.

Prefer `push_back` and indexed access for most uses.
When the final size is known in advance, `v.reserve(n)` avoids repeated **reallocations**.

The standard library

std::vector

```
#include <vector>

std::vector<int> v;
v.push_back(10);            // append
v.push_back(20);
v.push_back(30);

int first = v[0];           // 10, no bounds check
int safe  = v.at(1);        // 20, throws if out of range
std::size_t n = v.size();   // 3

v.pop_back();               // removes last element

```

Listing 6: Basic `std::vector` operations.

The standard library

Iterating containers

C++ containers expose **iterators** — objects that behave like pointers and allow traversal. Writing iterator types by hand is verbose; two modern features eliminate the boilerplate.

`auto` asks the compiler to **deduce the type** of a variable from its initializer — useful whenever the type is obvious from context or too verbose to write.

The **range-based** `for` loop iterates over any container without index arithmetic.

```
std::vector<int> v = {1, 2, 3, 4, 5};

for (auto x : v)            // x is a copy of each element
  std::cout << x << " ";

for (auto & x : v)          // x is a reference, no copy
  x *= 2;                   // modifies the vector in place

for (const auto & x : v)    // read-only reference
  std::cout << x << " ";

```

Listing 7: Iteration with `auto` and range-based `for`.

The standard library

Other essential containers

`std::array<T, N>` is a **fixed-size array** whose size is known at compile time.
Unlike a raw C array it knows its own size, can be copied, and works with standard algorithms.

`std::map<K, V>` is an **ordered associative container**: a sorted tree of key-value pairs with O(log n) lookup.

```
#include <array>
#include <map>

std::array<double, 3> point = {1.0, 2.0, 3.0};
std::cout << point[2] << std::endl;    // 3.0

std::map<std::string, int> scores;
scores["alice"] = 95;
scores["bob"]   = 87;

for (const auto & [key, val] : scores) // structured binding
  std::cout << key << ": " << val << std::endl;

```

Listing 8: `std::array` and `std::map` basics.

Roadmap

1 Templates

2 The standard library

3 Smart pointers

4 Lambdas and callbacks

Smart pointers

The problem with raw pointers

Raw pointers (`T *`) give full control over heap memory, but place the entire **burden of correctness** on the programmer:

- every `new` must be **matched** by exactly one `delete`;
- in the presence of **multiple exit paths**, exceptions, or callbacks, this matching becomes error-prone;
- **ownership** — who is responsible for deleting — is implicit and easy to lose track of.

In a callback-driven, multithreaded system

An object created in one thread, passed to a callback, and deleted somewhere else is a recipe for **use-after-free bugs** that are nearly impossible to reproduce and debug.

Smart pointers apply **RAII to heap memory**: the pointer object owns the resource and releases it automatically when destroyed.

Smart pointers

std::unique_ptr

`std::unique_ptr<T>` expresses **sole, exclusive ownership** of a heap-allocated object.

There is **exactly one** `unique_ptr` per object, the object is destroyed when the pointer goes out of scope.

This enforces at compile time that no two owners exist simultaneously.

Use it as the **default** choice when ownership is clear and singular.

It cannot be **copied**, only **moved** (see code example).

Smart pointers

std::unique_ptr

```
#include <memory>

{
  auto p = std::make_unique<int>(42); // preferred creation
  std::cout << * p << std::endl;      // 42

  auto q = std::move(p); // ownership transferred to q
                         // p is now empty (nullptr)
} // q goes out of scope: int is deleted automatically

```

Listing 9: unique_ptr creation, transfer, and automatic release.

Smart pointers

`std::shared_ptr`

`std::shared_ptr<T>` allows **shared ownership**: multiple pointers may refer to the same object.

It maintains a **reference count**: the object is **destroyed** only when the **last** `shared_ptr` to it is destroyed.

Always prefer `std::make_shared` over `new`: it allocates the **object** and the **control block** in a **single allocation**, and is **exception-safe**.

Smart pointers

std::shared_ptr

```
#include <memory>

auto a = std::make_shared<int>(7); // count = 1
{
  auto b = a;                      // count = 2
  std::cout << a.use_count() << std::endl; // 2
  auto c = a;                      // count = 3
} // b and c destroyed: count drops to 1

// a still valid; int alive
std::cout << a.use_count() << std::endl;   // 1
// a goes out of scope: count = 0, int destroyed

```

Listing 10: Reference counting with shared_ptr.

Smart pointers

std::weak_ptr

`std::weak_ptr<T>` is a **non-owning observer**: it refers to an object managed by a `shared_ptr` without incrementing the reference count.

It is used to **break ownership cycles** — two objects each holding a `shared_ptr` to the other would keep each other alive forever.

To access the object, a `weak_ptr` must be **promoted** to a `shared_ptr` first, which **may fail** if the object has already been destroyed.

Smart pointers

std::weak_ptr

```
std::shared_ptr<int> owner = std::make_shared<int>(99);
std::weak_ptr<int>   observer = owner;

if (auto p = observer.lock())   // returns shared_ptr or null
  std::cout << *p << std::endl; // object still alive

```

Listing 11: Promoting a weak_ptr to check if the object is still alive.

Smart pointers

The SharedPtr alias

In large codebases, writing `std::shared_ptr<MyClass>` everywhere is cumbersome.
A common C++ convention — and a **systematic pattern in ROS 2** — is to define a `SharedPtr` alias inside the class itself.

```
class MyNode
{
public:
  using SharedPtr = std::shared_ptr<MyNode>;
  // ...
};

```

Listing 12: The `SharedPtr` alias as used in ROS 2 classes.

You will encounter `Node::SharedPtr`, `Publisher::SharedPtr`, `Subscription::SharedPtr` and many others as soon as you open any rclcpp example.
They are all `std::shared_ptr` instantiations behind the alias.

Roadmap

1 Templates

2 The standard library

3 Smart pointers

4 Lambdas and callbacks

Lambdas and callbacks

Motivation

Many systems need to invoke **user-supplied code** at a point determined by the framework, not the user — a button press, a message arrival, a timer firing... on some **event**.
This pattern is called a **callback**.

In C, callbacks are plain **function pointers**: a fixed signature, no associated state, no closure. Passing context requires an extra `void *` argument and a cast — fragile and untyped.

C++ addresses this with two complementary tools:

- **lambda expressions**: anonymous functions that can **capture** local variables from their surrounding scope;
- `std::function`: a type-erased **wrapper** that can hold **any callable** — a lambda, a function pointer — with a given signature.

These two features are the backbone of event-driven programming in C++.
Every ROS 2 subscriber callback and timer callback is expressed through them.

A **lambda expression** defines an anonymous function inline, at the point of use, as a value.

```
// General form: [capture](params) -> return_type { body }
auto add = [](int a, int b) { return a + b; };
std::cout << add(3, 4) << std::endl; // 7

// Lambdas can be passed directly where a callable is expected
std::vector<int> v = {5, 1, 4, 2, 3};
std::sort(v.begin(), v.end(), [](int a, int b) {
  return a < b;
});

```

Listing 13: Lambda syntax and basic examples.

Lambdas are **first-class values**: they can be stored in variables, passed as arguments, and returned from functions.

Lambdas and callbacks

Capture

The **capture clause** `[...]` controls which **variables** from the **enclosing scope** are visible inside the lambda body, and how.

Capture by reference requires that the captured variable **outlives** the lambda — a common source of dangling-reference bugs.

```
int threshold = 10;

// Capture by value: lambda gets its own copy
auto by_val = [threshold](int x) { return x > threshold; };
// Capture by reference: lambda sees the original variable
auto by_ref = [& threshold](int x) { return x > threshold; };
threshold = 20; // by_ref will now use 20

// Inside a member function, capture [this] to access members
class Foo {
  int lim_ = 5;
  void bar() {auto baz = [this](int v) { return v < lim_; };}
};

```

Listing 14: Capture by value, by reference, and this.

Lambdas and callbacks

`std::function` and `std::bind`

The type of a lambda is **anonymous and compiler-generated** — it cannot be written explicitly. `std::function<R(Args...)>` is the solution: a **type-erased** wrapper that can hold **any callable** with a matching signature.

`std::bind` instead creates a callable by **binding** some (or all) arguments of an existing function or member function to fixed values.

It appears extensively in **existing C++ and ROS 2 code**.

Lambdas and callbacks

std::function and std::bind

```
#include <functional>

int plain_fn(int x) { return x * 2; }

std::function<int(int)> f;
f = plain_fn;                      // a plain function
f = [](int x) { return x * 2; }; // a lambda

// std::function objects can be stored in containers
std::vector<std::function<void(float)>> clbks;
clbks.push_back([](float v) { std::cout << v << std::endl; });
clbks.push_back([](float v) { /* log v */ });
for (auto & cb : clbks) cb(3.14f);

```

Listing 15: Storing heterogeneous callables in std::function.

Lambdas and callbacks

std::function and std::bind

```
#include <functional>

int add(int a, int b) { return a + b; }

// Bind first argument to 10; second remains free (_1)
auto add10 = std::bind(&add, 10, std::placeholders::_1);
std::cout << add10(5) << std::endl; // 15

// The modern equivalent with a lambda
auto add10_lambda = [](int b) { return add(10, b); };

```

Listing 16: std::bind compared to its lambda equivalent.